

MINI PROJECT TOPIC: REAL TIME WEATHER DASHBOARD

Step-by-Step Implementation:

Step 1: Create an IAM Role for EC2

Step 2: Write a User-Data Script

Step 3: Launch EC2 instance using AWS CLI

Step 4: Configure Security Group (HTTP + SSH)

Step 5: Access weather data in browser

Tools & Technologies Used for this project:

- **AWS EC2:** For hosting Python app
- **IAM:** Role management for EC2
- **Python:** Scripting & Web Server
- **Weather API:** For real-time data (e.g., OpenWeatherMap)
- **AWS CLI:** To configure everything via command prompt

Step 1: Create an IAM Role for EC2

The screenshot displays the AWS IAM console interface for the role 'weatherdashbaordrole'. The left sidebar shows the 'Identity and Access Management (IAM)' menu with a search bar containing 'weatherdashbaordrole' and a filter for 4 results. The main content area shows the role's summary, including its creation date (May 26, 2025, 16:40 UTC+05:30), last activity (3 hours ago), ARN, and instance profile ARN. Below the summary, the 'Permissions' tab is active, showing one attached policy: 'AmazonEC2FullAccess'. The policy is listed with a checkbox, policy name, type (AWS managed), and attached entities (1).

weatherdashbaordrole [Info](#) [Delete](#)

Allows EC2 instances to call AWS services on your behalf.

Summary [Edit](#)

Creation date
May 26, 2025, 16:40 (UTC+05:30)

ARN
[arn:aws:iam::158464670849:role/weatherdashbaordrole](#)

Instance profile ARN
[arn:aws:iam::158464670849:instance-profile/weatherdashbaordrole](#)

Last activity
[3 hours ago](#)

Maximum session duration
1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (1) [Info](#) [Simulate](#) [Remove](#) [Add permissions](#)

You can attach up to 10 managed policies.

Filter by Type

☐	Policy name ?	Type	Attached entities
☐	AmazonEC2FullAccess	AWS managed	1

Weather API :

A **weather API** is a service that provides up-to-date weather information via the internet.

You register for a free account, and they give you an **API key**, which you include in your code to access their data.

Step 2: Write the EC2 User-Data Script to be used in command prompt

```
from flask import Flask, render_template_string, request
import requests

app = Flask(__name__)

API_KEY = "b441919c5a3f277206248acb19739a05"

HTML = '''
<!DOCTYPE html>
<html>
<head><title>Weather Dashboard</title></head>
<body>
  <h2>Enter City Name:</h2>
  <form method="POST">
    <input type="text" name="city" required>
    <button type="submit">Get Weather</button>
  </form>

  {% if weather %}
    <h3>Weather in {{ weather['name'] }}:</h3>
    <p>Temperature: {{ weather['main']['temp'] }}°C</p>
    <p>Condition: {{ weather['weather'][0]['description'] }}</p>
  {% endif %}
</body>
</html>
'''

@app.route('/', methods=['GET', 'POST'])
def home():
    weather = None
    if request.method == 'POST':
        city = request.form['city']
        url = f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}&units=metric"
        response = requests.get(url)
        if response.status_code == 200:
            weather = response.json()
        return render_template_string(HTML, weather=weather)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=80)
```

Sample Response

```
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing                :                               1/1
  Running scriptlet: nginx-filesystem-1:1.26.3-1.amzn2023.0.1.noarch 1/6
  Installing         : nginx-filesystem-1:1.26.3-1.amzn2023.0.1.noarch 1/6
  Installing         : nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch 2/6
  Installing         : libunwind-1.4.0-5.amzn2023.0.2.x86_64 3/6
  Installing         : gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64 4/6
  Installing         : nginx-core-1:1.26.3-1.amzn2023.0.1.x86_64 5/6
  Installing         : nginx-1:1.26.3-1.amzn2023.0.1.x86_64 6/6
  Running scriptlet: nginx-1:1.26.3-1.amzn2023.0.1.x86_64 6/6
  Verifying          : gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64 1/6
  Verifying          : libunwind-1.4.0-5.amzn2023.0.2.x86_64 2/6
  Verifying          : nginx-1:1.26.3-1.amzn2023.0.1.x86_64 3/6
  Verifying          : nginx-core-1:1.26.3-1.amzn2023.0.1.x86_64 4/6
  Verifying          : nginx-filesystem-1:1.26.3-1.amzn2023.0.1.noarch 5/6
  Verifying          : nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch 6/6

Installed:
gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64      libunwind-1.4.0-5.amzn2023.0.2.x86_64
nginx-1:1.26.3-1.amzn2023.0.1.x86_64            nginx-core-1:1.26.3-1.amzn2023.0.1.x86_64
nginx-filesystem-1:1.26.3-1.amzn2023.0.1.noarch  nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch

Complete!
[ec2-user@ip-172-31-14-140 ~]$ sudo systemctl start nginx
[ec2-user@ip-172-31-14-140 ~]$ sudo systemctl enable nginx
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr/lib/systemd/system/nginx.service.
[ec2-user@ip-172-31-14-140 ~]$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Mon 2025-05-26 16:41:24 UTC; 24s ago
     Main PID: 30742 (nginx)
       Tasks: 2 (limit: 1111)
      Memory: 2.5M
         CPU: 42ms
      CGroup: /system.slice/nginx.service
              └─30742 "nginx: master process /usr/sbin/nginx"
                └─30743 "nginx: worker process"
```

Step 3: Launch EC2 Instance Using AWS CLI

The screenshot shows the AWS Management Console interface for EC2 instances. On the left, there is a navigation menu with options like 'Instances', 'Instance Types', 'Launch Templates', 'Spot Requests', 'Savings Plans', 'Reserved Instances', 'Dedicated Hosts', 'Capacity Reservations', 'Images', 'Elastic Block Store', and 'Network & Security'. The main area displays a list of instances under the heading 'Instances (1/2) Info'. Two instances are listed: 'weather dashb...' (Terminated) and 'weatherapp' (Running). The 'weatherapp' instance is selected, and its details are shown below. The details include the Instance ID 'i-057cd3869c7b088f7', Public IPv4 address, Private IPv4 addresses, Instance state (Terminated), Public DNS, and Hostname type.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
weather dashb...	i-057cd3869c7b088f7	Terminated	t2.micro	-	View alarms +	eu-west-3a	-
weatherapp	i-07916458f8dacc6fa	Running	t2.micro	2/2 checks passed	View alarms +	eu-west-3a	ec2-15-23

i-057cd3869c7b088f7 (weather dashboard)

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

Instance summary Info

Instance ID	Public IPv4 address	Private IPv4 addresses
i-057cd3869c7b088f7	-	-

IPv6 address	Instance state	Public DNS
-	Terminated	-

Hostname type

Response after launching EC2



i-07916458f8dacc6fa (weatherapp)

PublicIPs: 15.237.177.17 PrivateIPs: 172.31.14.140

Step 4: Configure Security Group

- We made sure our EC2 instance is **accessible from the internet**.
- In the **Security Group**, we opened:
 - **Port 22**: For SSH access (to connect to the instance)
 - **Port 80**: For HTTP access (to view weather data in browser)
- This allows users to visit the EC2 public IP and see weather info.

EC2 > Security Groups > sg-0720c935c5d1deb18 - launch-wizard-1

EC2

- Dashboard
- EC2 Global View
- Events
- ▼ Instances
 - Instances
 - Instance Types
 - Launch Templates
 - Spot Requests
 - Savings Plans
 - Reserved Instances
 - Dedicated Hosts
 - Capacity Reservations
- ▼ Images
 - AMIs
 - AMI Catalog
- ▼ Elastic Block Store
 - Volumes

sg-0720c935c5d1deb18 - launch-wizard-1

Actions

Details

Security group name launch-wizard-1	Security group ID sg-0720c935c5d1deb18	Description launch-wizard-1 created 2025-05-26T15:19:10.759Z	VPC ID vpc-04295001e0eae7ee9
Owner 158464670849	Inbound rules count 2 Permission entries	Outbound rules count 1 Permission entry	

[Inbound rules](#) | [Outbound rules](#) | [Sharing - new](#) | [VPC associations - new](#) | [Tags](#)

Inbound rules (2) [Manage tags](#) [Edit inbound rules](#)

Search

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range
☐	-	sgr-08dfa1cf3a190f362	IPv4	HTTP	TCP	80
☐	-	sgr-003de4e36f65c5b53	IPv4	SSH	TCP	22

Step 5: Access the Real-Time Weather Dashboard

http://<EC2-IP>/weather

← ↻ ⚠ Not secure 15.237.177.17

Click to go back, hold to see history

Current Weather in London

Temperature: 16.12 Å°C

Humidity: 67 %

Description: broken clouds

Hello from Nginx root /var/www/html